

Relatório Técnico: Análise de Similaridade de Comentários do YouTube com TF-IDF

Gustavo Nascimento Geminiano

4 de novembro de 2025

1 Introdução

Este relatório descreve o desenvolvimento de um script em Python que utiliza a API do YouTube para coletar comentários de um vídeo e calcular a similaridade entre esses comentários e uma *query* informada pelo usuário. O método de similaridade adotado baseia-se na vetorização TF-IDF e na métrica de cosseno.

2 Objetivos

- Baixar comentários de vídeos do YouTube automaticamente;
- Pré-processar texto em português utilizando o `nltk`;
- Aplicar o modelo TF-IDF para transformar texto em vetores numéricos;
- Calcular a similaridade entre uma consulta e os comentários;
- Exibir os comentários mais relevantes de acordo com o grau de similaridade.

3 Metodologia

O código foi implementado em Python e faz uso das bibliotecas:

- `nltk`: para tratamento de linguagem natural e remoção de stopwords;
- `googleapiclient`: para conexão com a API do YouTube;
- `sklearn`: para vetorização TF-IDF e cálculo da similaridade do cosseno.

4 Código-Fonte

```
import nltk
from googleapiclient.discovery import build
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import time
import random

nltk.download('stopwords')
from nltk.corpus import stopwords
stopwords_pt = stopwords.words('portuguese')

api_key = "SUA_API_KEY"
link = input("Qual link do seu vídeo? ")
video_id = link.replace("https://www.youtube.com/watch?v=",
                        "", "")
query = [input("Qual sua query? ")]

youtube = build("youtube", "v3", developerKey=api_key)
comentarios = []
next_page_token = None

print("Baixando comentários...")
while True:
    try:
        request = youtube.commentThreads().list(
            part="snippet,replies",
            videoId=video_id,
            maxResults=100,
            pageToken=next_page_token,
            textFormat="plainText"
        )
        response = request.execute()

        for item in response["items"]:
            comentario = item["snippet"]["topLevelComment"]
            ["snippet"]["textDisplay"]
            comentarios.append(comentario)
            if "replies" in item:
                for reply in item["replies"]["comments"]:
```

```

        comentarios.append(reply["snippet"]["
            textDisplay"])

    next_page_token = response.get("nextPageToken")
    if not next_page_token:
        break
    time.sleep(random.uniform(0.5, 1.5))
except Exception as e:
    print(f"Erro: {e}")
    time.sleep(5)
    continue

comentarios_limpos = [" ".join([p for p in c.lower().
    split() if p not in stopwords_pt]) for c in
    comentarios]
query_limpa = [" ".join([p for p in q.lower().split() if
    p not in stopwords_pt]) for q in query]

vectorizer = TfidfVectorizer()
tfidf_docs = vectorizer.fit_transform(comentarios_limpos)
tfidf_query = vectorizer.transform(query_limpa)

similaridades = cosine_similarity(tfidf_query, tfidf_docs
    )[0]
comentarios_ordenados = sorted([(c, s) for c, s in zip(
    comentarios, similaridades) if s > 0], key=lambda x: x
    [1], reverse=True)

if comentarios_ordenados:
    for i, (comentario, score) in enumerate(
        comentarios_ordenados[:5], 1):
        print(f"{i}. {score:.3f} {comentario}")
else:
    print("Nenhum comentário relevante encontrado.")

```

5 Adendos e Limitações

Durante o desenvolvimento e execução dos testes, foi identificado que o script não é capaz de coletar 100% dos comentários de determinados vídeos. Essa limitação não está relacionada ao código em si, mas a restrições impostas pela própria **API do YouTube Data**. Em alguns casos, a API não retorna

todas as páginas de resultados, principalmente em vídeos com um volume muito alto de comentários ou com respostas aninhadas em múltiplos níveis. Adicionalmente, existem limites de taxa (*rate limits*) que restringem o número de requisições possíveis em um determinado intervalo de tempo. Como consequência, o total de comentários baixados pode ser inferior ao número exibido publicamente no vídeo. Apesar disso, a amostra coletada é suficientemente representativa para análises baseadas em similaridade textual.

6 Resultados

Os resultados consistem em uma lista dos comentários mais semelhantes à consulta, classificados por grau de similaridade. Essa abordagem permite identificar rapidamente respostas ou discussões relevantes em vídeos com grande volume de comentários.

7 Conclusão

O sistema desenvolvido demonstra a aplicabilidade de técnicas de Processamento de Linguagem Natural (PLN) para análise de conteúdo textual em redes sociais. Possíveis expansões incluem a análise de sentimento e a visualização dos resultados.